Project - High Level Design on Real Estate AI Platform

Course Name: Devops

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	HARSHIT MITTAL	EN22CS301401
2.	HARSHAL DESHMUKH	EN22CS301396
3.	HARSHIT CHOUDHARY	EN22CS301400
4.	SALONEE CHOUHAN	EN23CS3L1017
5.	HARSHIT SHARMA	EN22CS301403
6.	HARSHIT BHARDWAJ	EN22CS301399

Group Name: Group 11D4

Project Number: DO-47

Industry Mentor Name: Mr. Vaibhav Sir

University Mentor Name: Prof. Avnesh Kumar Joshi

Academic Year: 2025-26

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. Data Design.
   3.1. Data Model
   3.2. Data Access Mechanism
   3.3. Data Retention Policies
   3.4. Data Migration

4. Interfaces
   4.1. User Interface
   4.2. Admin Interface
   4.3. API Interface

5. State and Session Management

6. Caching

7. Non-Functional Requirements
   7.1. Security Aspects
   7.2. Performance Aspects

8. References

High-Level Design (HLD) Document: AI-Powered Real Estate Platform

1. Introduction

1.1. Scope of the Document

This document provides the High-Level Design for the AI-Powered Real Estate Platform. It defines the system architecture, component interactions, data management strategies, and deployment infrastructure. This covers the full-stack microservices design, the Retrieval-Augmented Generation (RAG) AI integration, and the end-to-end DevOps CI/CD pipeline.

1.2. Intended Audience

This document is intended for Software Developers, DevOps Engineers, System Architects, Database Administrators, and Project Stakeholders involved in the development, deployment, and maintenance of the platform.

1.3. System Overview

The system is a cloud-based real estate platform that allows users to browse property listings and interact with an intelligent AI assistant. The platform uses a microservices architecture with a React frontend and a FastAPI backend. It leverages PostgreSQL for relational data and ChromaDB for vector-based semantic search. The AI assistant is powered by Google Gemini and uses a RAG pipeline to ensure responses are grounded in actual property data. The infrastructure is fully automated using Terraform, Ansible, Jenkins, Docker, and Kubernetes on AWS.

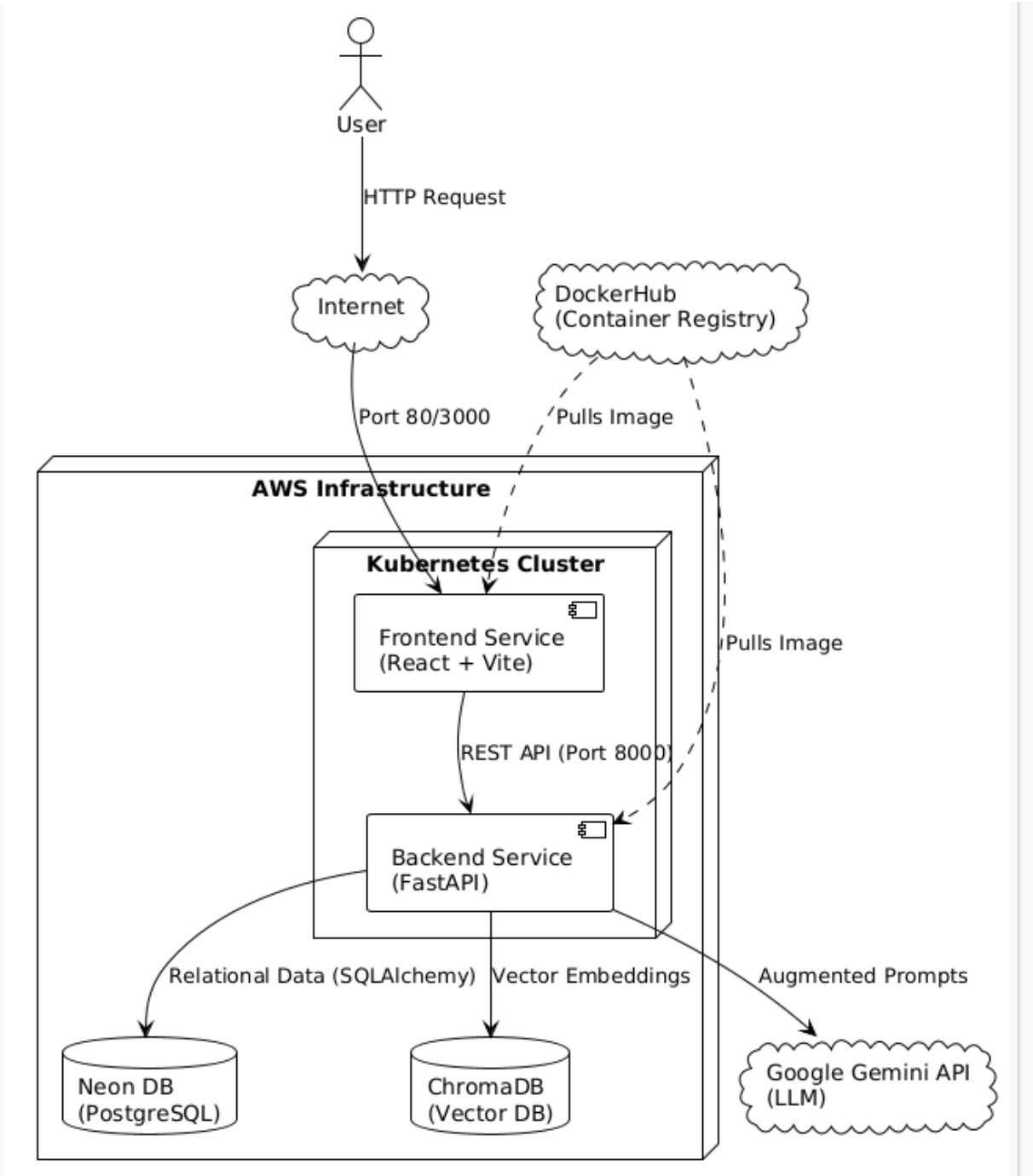
2. System Design

2.1. Application Design

The application follows a containerized microservices design pattern.

- **Frontend Service:** Built with React and Vite, running on port 3000, responsible for UI rendering and client-side logic.
- **Backend Service:** Built with FastAPI, running on port 8000, handling business logic, API routing, and AI orchestration.
- **Databases:** A hybrid approach using Neon DB (PostgreSQL) for transactional data and ChromaDB for AI vector embeddings.
- **Infrastructure:** Provisioned via Terraform on AWS EC2, configured via Ansible, and orchestrated using Kubernetes.

System Architecture Diagram (Component Diagram)



2.2. Process Flow

Deployment Process Flow:

1. Developer pushes code to the GitHub repository.
2. Jenkins CI/CD pipeline triggers an automated build.
3. Docker images (realestate-frontend, realestate-backend) are built and pushed to DockerHub.
4. Terraform provisions AWS EC2 infrastructure; Ansible configures the server and Docker environment.
5. Kubernetes deploys the updated containers into Pods, making the application live.

2.3. Information Flow

AI Query Information Flow (RAG Pipeline):

1. User submits a query via the React Frontend.
2. The FastAPI backend receives the request and passes it to the Embedding Model (Sentence Transformers).
3. The embedding is used to search the Vector Database (ChromaDB) for semantically relevant property data.
4. Retrieved property data is appended to the user query as context.
5. The combined prompt is sent to the Google Gemini LLM API.
6. The generated, fact-based response is returned to the user interface.

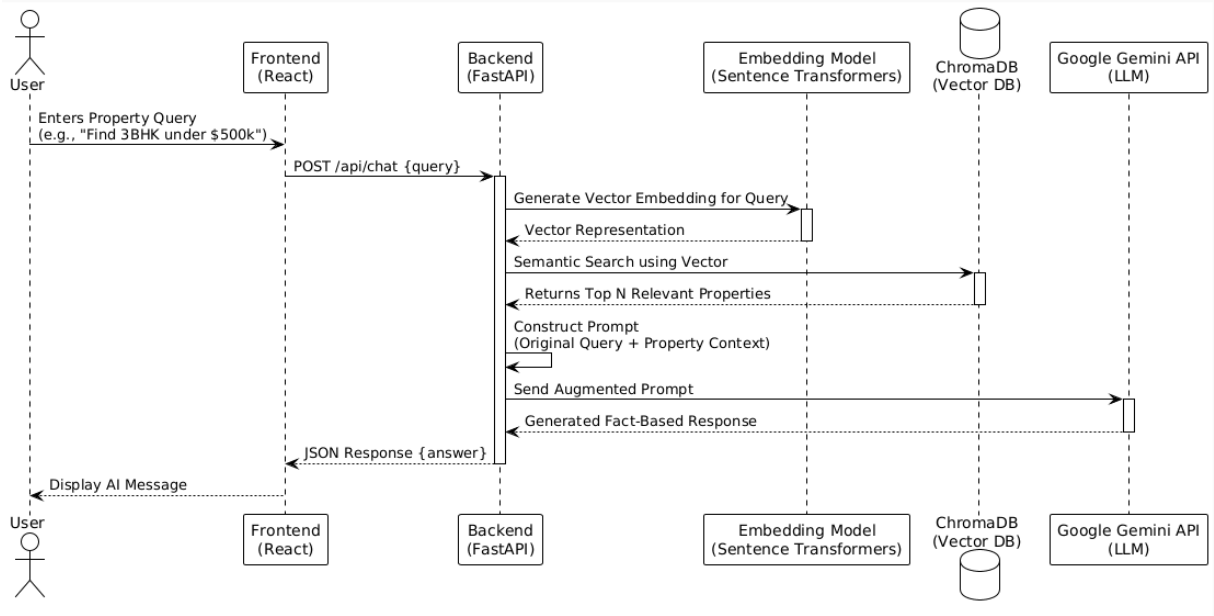
2.4. Components Design

- **Web UI Component:** React application utilizing Axios for asynchronous HTTP API calls.
- **API Gateway/Server:** FastAPI application serving RESTful endpoints.
- **AI Engine:** Integrates LangChain and CrewAI to manage prompts and orchestration between Sentence Transformers and the Gemini API.
- **Orchestration Component:** Kubernetes managing Pods, Deployments, and Services for high availability and load balancing.

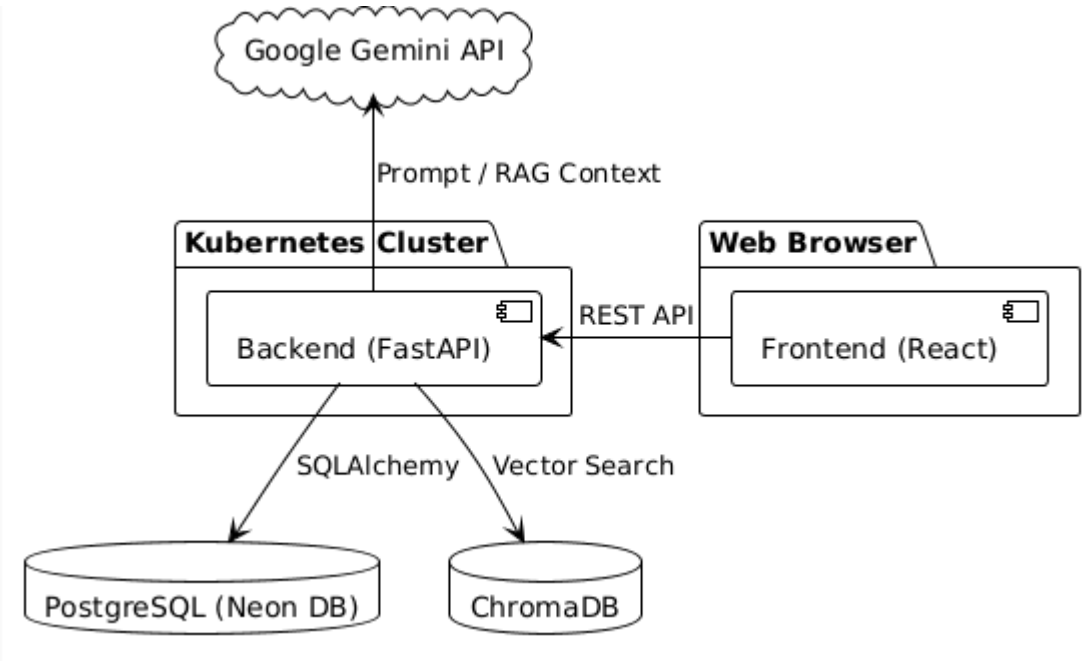
2.5. Key Design Considerations

- **Data Accuracy:** Implementation of the RAG pattern strictly enforces that the AI models answer based on database properties, preventing hallucinations.
- **Scalability:** Utilizing Kubernetes allows the frontend and backend services to scale independently based on load.
- **Infrastructure as Code (IaC):** Using Terraform and Ansible ensures that environments are reproducible and configuration drift is minimized.

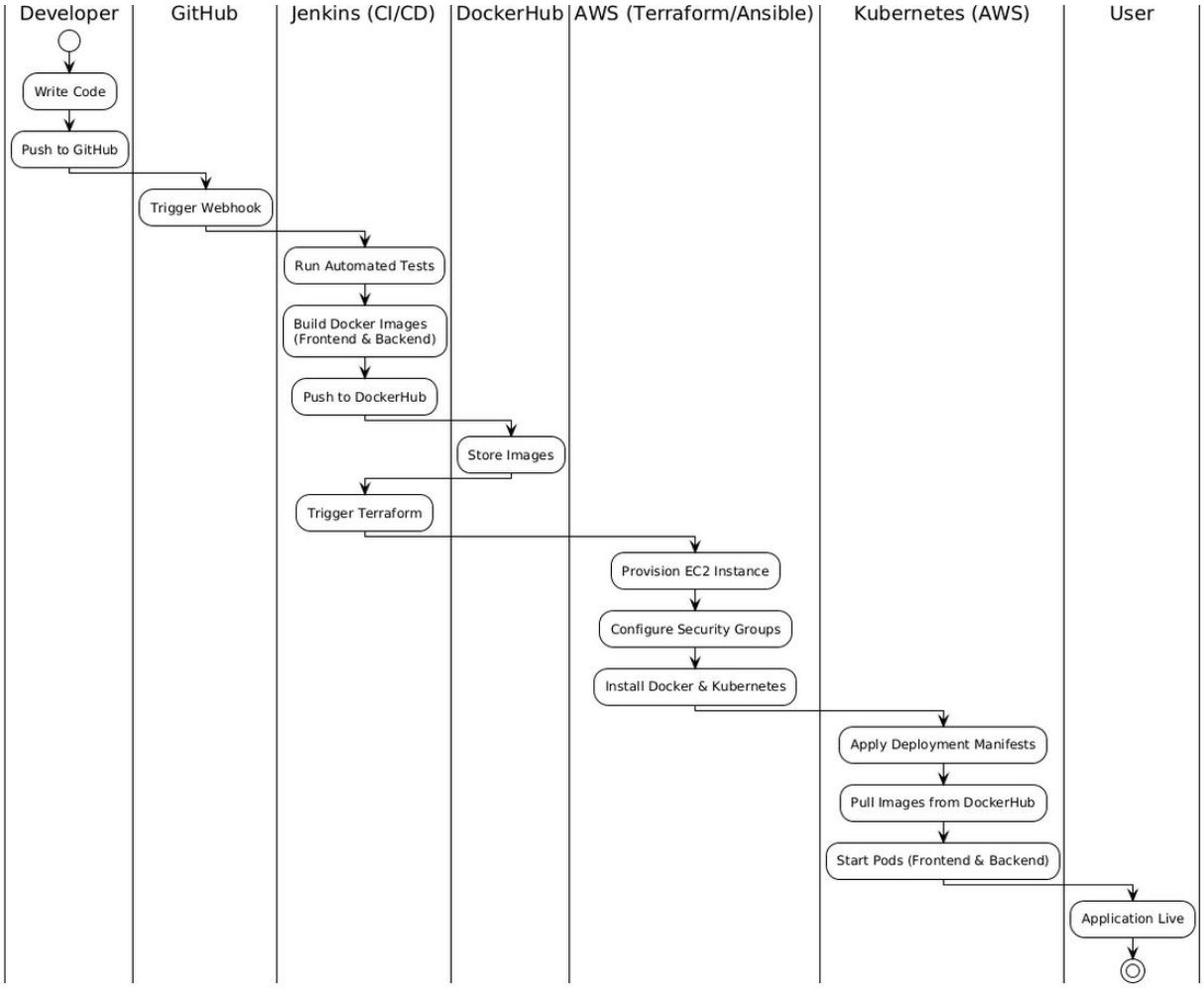
AI RAG Processing Data Flow (Sequence Diagram)



Component Design



DevOps CI/CD Pipeline (Activity Diagram)



2.6. API Catalogue

While specific endpoints will be detailed in the Low-Level Design, the core API categories include:

- **Authentication:** Endpoints for user signup, login, and JWT token generation.
- **Property Management:** CRUD operations for real estate listings.
- **AI Integration:** POST endpoints to receive user text queries and return LLM-generated responses.
- Here is the API Catalogue for your AI-Powered Real Estate Platform. Since your backend is built with FastAPI, these endpoints represent the RESTful structure that would be automatically documented in your Swagger UI (`http://[EC2-IP]:8000/docs`).
- This catalogue is divided into four core modules: **Authentication**, **Property Management**, **Appointments**, and **AI Assistant**.

- **1. Authentication & Users**
- These endpoints handle user onboarding, login, and profile management using JSON Web Tokens (JWT).

Method	Endpoint	Description	Auth Required	Role
POST	`/api/auth/register`	Creates a new user account and hashes the password.	No	Public
POST	`/api/auth/login`	Authenticates a user and returns a JWT token.	No	Public
GET	`/api/auth/me`	Retrieves the profile details of the currently logged-in user.	Yes	Any

- ---
- **2. Property Management**
- These endpoints handle the core CRUD (Create, Read, Update, Delete) operations for real estate listings.

Method	Endpoint	Description	Auth Required	Role
GET	`/api/properties`	Retrieves a list of properties. Supports query parameters for filtering (e.g., `?min_price=500000&bedrooms=3`).	No	Public
GET	`/api/properties/{id}`	Retrieves detailed information for a specific property by its UUID.	No	Public
POST	`/api/properties`	Creates a new property listing and generates its semantic embedding in ChromaDB.	Yes	Admin

Method	Endpoint	Description	Auth Required	Role
PUT	`/api/properties/{id}`	Updates an existing property's details and synchronizes the ChromaDB embedding.	Yes	Admin
DELETE	`/api/properties/{id}`	Deletes a property from PostgreSQL and removes its vector from ChromaDB.	Yes	Admin

- ---
- **3. Appointments**
- These endpoints allow users to schedule viewings and admins to manage them.

Method	Endpoint	Description	Auth Required	Role
POST	`/api/appointments`	Books a new viewing appointment for a specific property.	Yes	Customer
GET	`/api/appointments`	Retrieves appointments. Customers see their own; Admins see all.	Yes	Any
PUT	`/api/appointments/{id}/status`	Updates appointment status (e.g., from 'pending' to 'confirmed').	Yes	Admin

Method	Endpoint	Description	Auth Required	Role
DELETE	`/api/appointments/{id}`	Cancels an appointment.	Yes	Customer/Admin

- ---
- **4. AI Assistant (RAG Pipeline)**
- This is the core endpoint where the frontend React app communicates with the LangChain/Gemini backend.

Method	Endpoint	Description	Auth Required	Role
POST	`/api/chat`	Accepts a user's text query, performs a vector search in ChromaDB, and returns a Gemini-generated response based on the retrieved property data.	Yes	Any

- **Example Payload for `/api/chat`:**
- JSON
- {
- "query": "I am looking for a 3-bedroom house under $600,000 near the downtown area."
- }
- **Example Response:**
- JSON
- {
- "response": "I found 2 properties matching your criteria. The first is a 3-bedroom townhouse downtown listed at $550,000...",
- "sources": ["uuid-of-property-1", "uuid-of-property-2"]
- }

3. Data Design

3.1. Data Model

The system relies on two distinct data models:

- **Relational Model (PostgreSQL):** Stores structured entities including Users, Properties, and Appointments.
- **Vector Model (ChromaDB):** Stores high-dimensional semantic embeddings of property details used for similarity searches by the AI.

3.2. Data Access Mechanism

- **Relational Data:** The backend utilizes **SQLAlchemy** as the Object-Relational Mapper (ORM) to securely and efficiently query the Neon PostgreSQL database.
- **Vector Data:** Accessed via the native ChromaDB client integrated within the LangChain pipeline for retrieval operations.

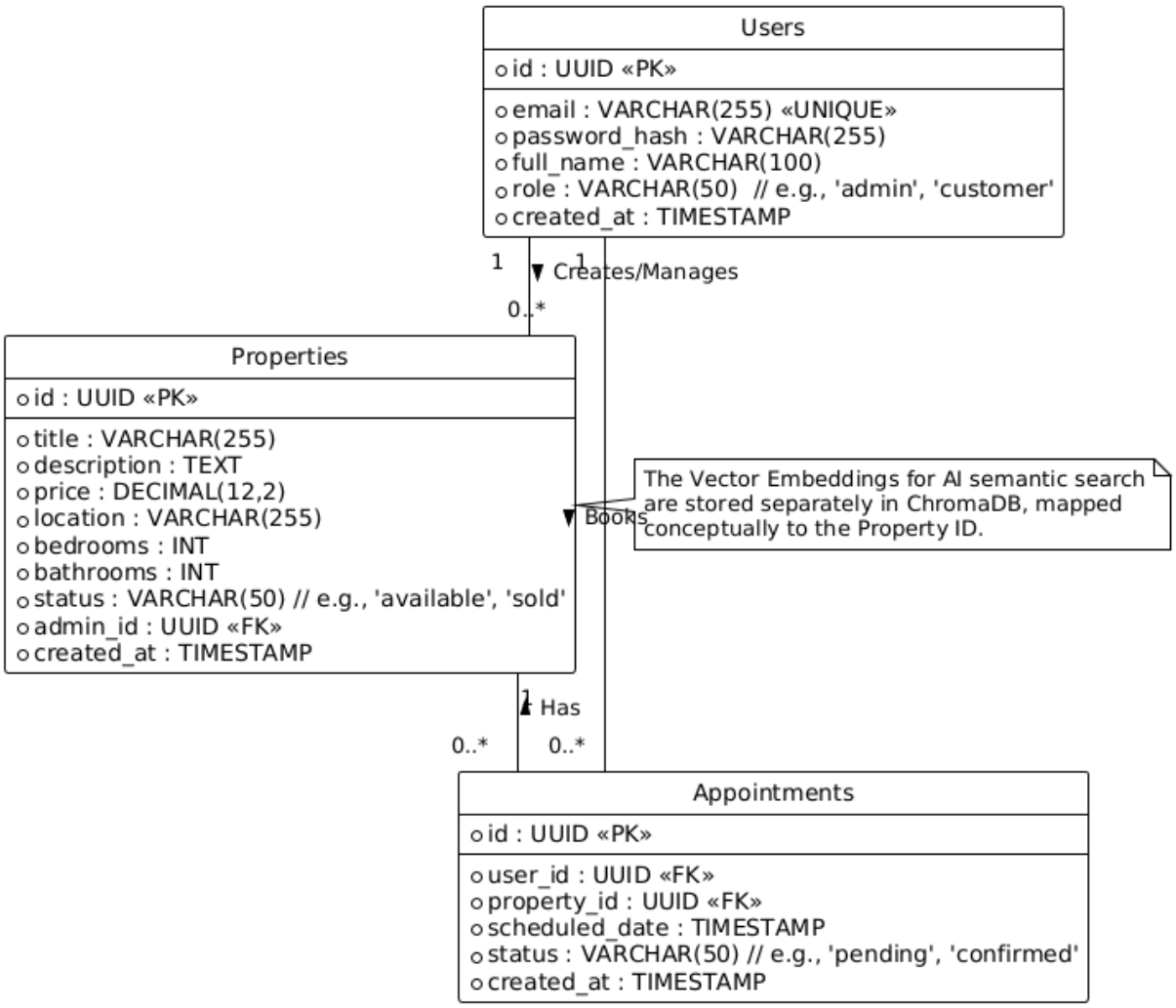
3.3. Data Retention Policies

- **Active Properties:** Retained indefinitely until explicitly removed or marked as sold by an admin.
- **AI Chat Logs:** Session-based chat histories are retained temporarily in memory or lightweight session storage to maintain conversation context, adhering to standard privacy practices.

3.4. Data Migration

Database schema changes and migrations for the PostgreSQL database are managed programmatically to ensure consistency across environments during CI/CD deployments.

E-R Diagram



4. Interfaces

4.1. User Interface

A responsive web interface built with React. It provides user authentication flows, searchable property listing grids, property detail pages, and an integrated chat window for communicating with the real estate AI assistant.

4.2. Admin Interface

A secure, gated section of the React frontend available only to authenticated administrators. It provides property management capabilities, allowing admins to add, update, and remove property listings from the database.

4.3. API Interface

The backend exposes a RESTful API over HTTP (port 8000). Communication between the frontend and backend utilizes JSON payloads. The API is documented automatically via FastAPI's built-in Swagger/OpenAPI UI integration accessible at http://[EC2-IP]:8000/docs.

5. State and Session Management

- **Authentication State:** Managed via **JWT (JSON Web Tokens)**. Upon successful login, the backend issues a JWT, which the React frontend stores securely and attaches to subsequent Axios API requests.
- **Application State:** The React frontend manages UI state, while the FastAPI backend remains entirely stateless to allow seamless horizontal scaling within the Kubernetes cluster.

6. Caching

- **Static Assets:** The React production build served by the serve static server (or Kubernetes Ingress) leverages standard browser caching for CSS, JS, and image assets.
- **Semantic Search:** ChromaDB intrinsically optimizes vector search performance by maintaining data in memory-mapped files, acting as a highly efficient retrieval layer for the AI pipeline.

7. Non-Functional Requirements

7.1. Security Aspects

- **Authentication:** All sensitive API routes are protected via JWT validation.
- **Secret Management:** Database URLs, JWT secrets, and the Gemini API Key are never hardcoded. They are stored securely in .env files and injected into Kubernetes Pods as Kubernetes Secrets.
- **Network Security:** AWS Security Groups restrict inbound traffic exclusively to necessary ports (22 for SSH, 80/3000 for Frontend, 8000 for Backend APIs). Container isolation provided by Docker prevents cross-service interference.

7.2. Performance Aspects

- **AI Latency:** The RAG architecture separates the fast vector search (ChromaDB) from the external API call (Gemini) to optimize response times.

- **Resource Efficiency:** Deployed on an AWS t2.micro (Ubuntu) instance to stay within free-tier limits during initial rollout, with Kubernetes ready to orchestrate additional resources when scaling is required.

8. References

- System Architecture & CI/CD Documentation
- LLD: AI-Powered Real Estate Platform with DevOps Pipeline
- Kubernetes Deployment YAML Specifications